



CIS 678 Machine Learning

Course Review Week (Part 1)

Proximity or distance metric

- kNN model:**

- Distance based
- Can be applied to both Regression and Classification tasks

[illegible]

Data/Feature Encoding

Given a Data Table

Age (Yr)	Height (M)	Weight (Kg)	Sex	Color preference
10	1.3	45	F	Red
30	1.7	67	M	Blue
65	1.5	57	F	Green
...	...	...	...	...

- We are saying, Green is closer to Red than Blue,
- And enforcing Bias in the Vector Space
- Which doesn't seem ok.

{R, G, B}

{R=0, G=1, B=2}

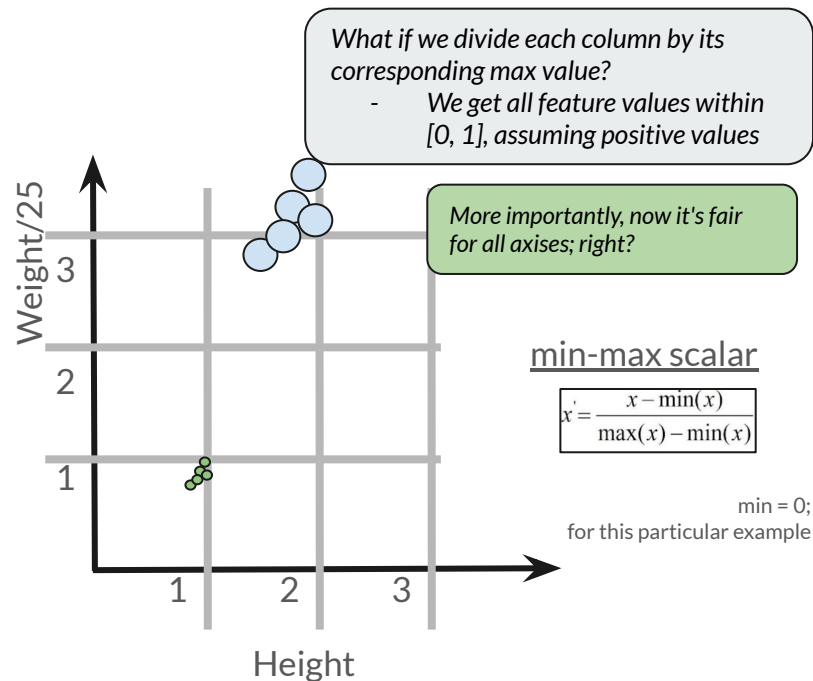
Do you see any issue here?

Label encoder

Feature Scaling

Lets we are given a set of data points; we want to plot them on a 2D cartesian plane (vector space)

	height(meter)	weight(kg)	weight(kg)/25	height/max(height)	weight/max(weight)
0	1.50	70	2.80	0.83	0.85
1	1.70	80	3.20	0.94	0.98
2	1.80	82	3.28	1.00	1.00
3	1.60	75	3.00	0.89	0.91
4	1.75	78	3.12	0.97	0.95





Feature Scaling

We have other scaling as well:

- Standard scalar

	height(meter)	weight(kg)	height_ss	weight_ss
0	1.50	70	1.58	1.67
1	1.70	80	-0.28	-0.72
2	1.80	82	-1.21	-1.19
3	1.60	75	0.65	0.48
4	1.75	78	-0.74	-0.24

$$z = \frac{x - \mu}{\sigma}$$

μ = Mean

σ = Standard Deviation

- Scaled features are centered across the Zero-axis
- NN community uses this extensively

1.1 Learning Objectives

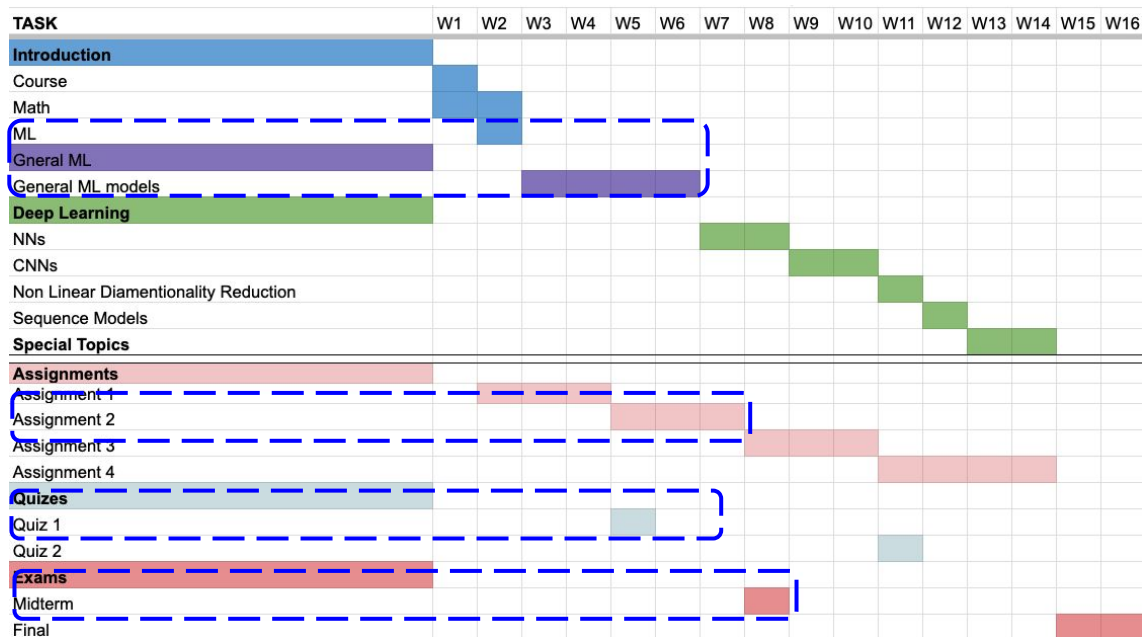
- Implement common distance metrics used in machine learning
- Analyze how distance measures affect similarity-based models
- Apply k-Nearest Neighbors for regression tasks
- Interpret empirical results using summary statistics and visualizations

You are given a dataset of residential properties. Your goal is to identify houses that are most similar to a given property based on their feature representations. Distance metrics play a central role in similarity-based learning methods such as k-Nearest Neighbors (k-NN).

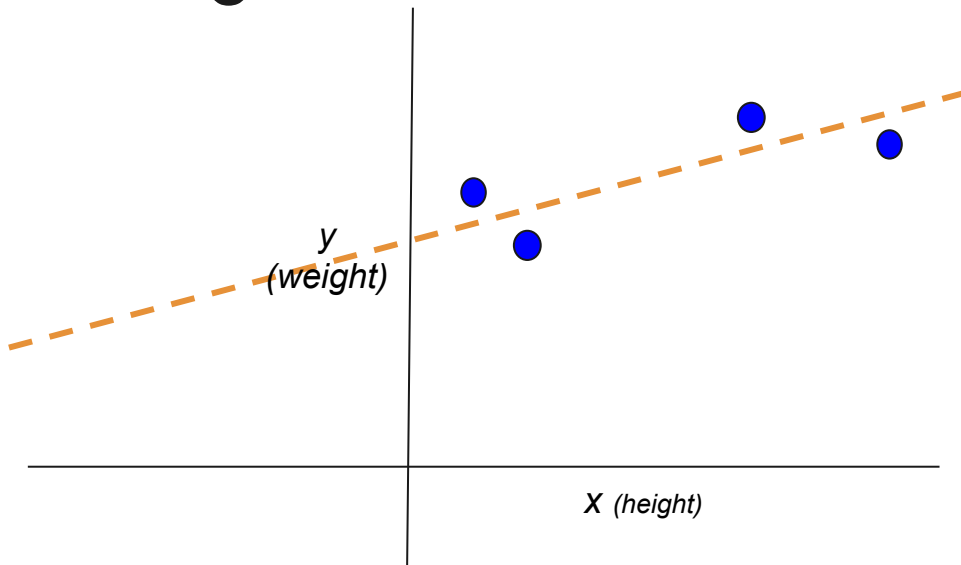
Data file: https://github.com/mdkamrulhasan/data-public/blob/main/house_prices.csv

[illegible]





Linear Regression



Model

$$\hat{y} = \beta_0 + \beta_1 x$$

$$\Theta = \{\beta_0, \beta_1\}$$

Fitting Error

$$\epsilon = |\hat{y} - y|$$

Optimization function

$$E_{\Theta} = \frac{1}{2} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

$$\Theta^* = \operatorname{argmin}_{\Theta} E\{(x_i, y_i)\}_{i=1, \dots, N}$$

Linear to Polynomial Regression

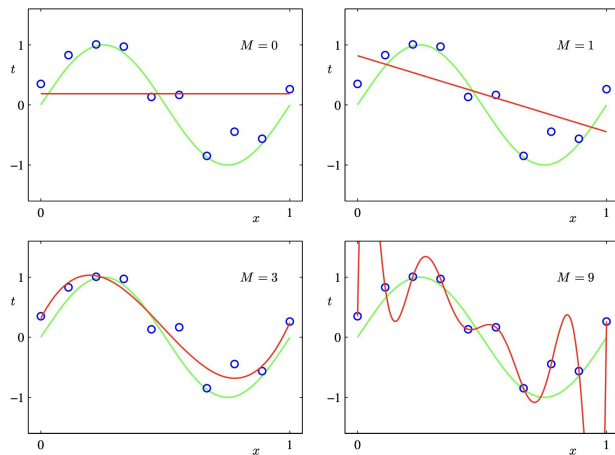


Table 1.1 Table of the coefficients w^* for polynomials of various order. Observe how the typical magnitude of the coefficients increases dramatically as the order of the polynomial increases.

	$M = 0$	$M = 1$	$M = 6$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

Model generalization

Linear to Polynomial Regression

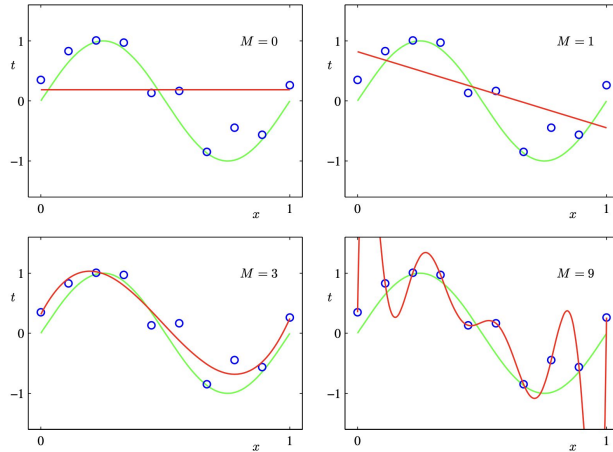


Table 1.1 Table of the coefficients w^* for polynomials of various order. Observe how the typical magnitude of the coefficients increases dramatically as the order of the polynomial increases.

	$M = 0$	$M = 1$	$M = 6$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

Absolute values
are increasing

Model generalization

Linear to Polynomial Regression

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2$$

Essentially, the same formulation

Generally ML vs Math conventions

$$W^* = \operatorname{argmin}_W E\{(x_i, t_i)\}_{i=1, \dots, N}$$

Model

$$\hat{y} = \beta_0 + \beta_1 x$$

$$\Theta = \{\beta_0, \beta_1\}$$

$$\epsilon = |\hat{y} - y|$$

Optimization function

$$E_{\Theta} = \frac{1}{2} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

$$\Theta^* = \operatorname{argmin}_{\Theta} E\{(x_i, y_i)\}_{i=1, \dots, N}$$

Linear to Polynomial Regression

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2$$

Regularizer

$$\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{1}{2} \|\mathbf{w}\|^2$$

Model generalization: Regularization

Table 1.1 Table of the coefficients $\mathbf{w}^*$ for polynomials of various order. Observe how the typical magnitude of the coefficients increases dramatically as the order of the polynomial increases.

	$M = 0$	$M = 1$	$M = 6$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

Abs values
Are increasing

Linear to Polynomial Regression

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2$$

Table 1.1 Table of the coefficients $\mathbf{w}^*$ for polynomials of various order. Observe how the typical magnitude of the coefficients increases dramatically as the order of the polynomial increases.

	$M = 0$	$M = 1$	$M = 6$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

$$\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{1}{2} \|\mathbf{w}\|^2$$



How to control this?

decreasing

Model generalization: Regularization

Linear to Polynomial Regression

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2$$

Hyperparameter

$$\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

$\|\mathbf{w}\|^2$

Model generalization: Regularization

Table 1.1 Table of the coefficients $\mathbf{w}^*$ for polynomials of various order. Observe how the typical magnitude of the coefficients increases dramatically as the order of the polynomial increases.

	$M = 0$	$M = 1$	$M = 6$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

Who to control this?



Probabilistic Twin

Of

The Least Squares Solution:

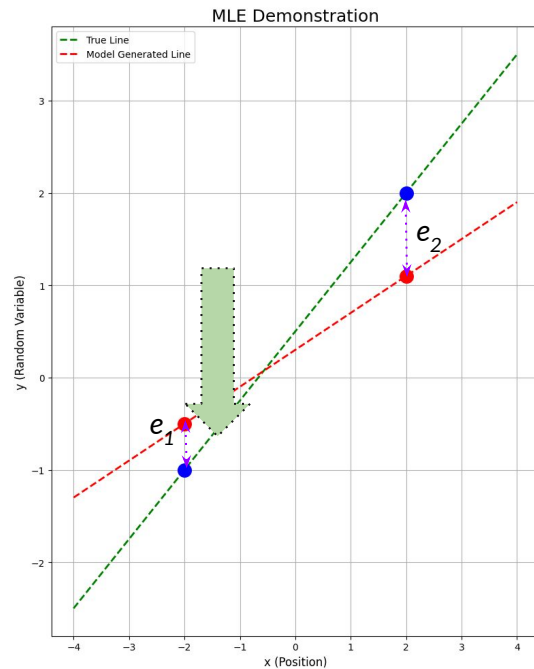
$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2$$

Probabilistic Twin

Of

The Least Squares Solution:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2$$



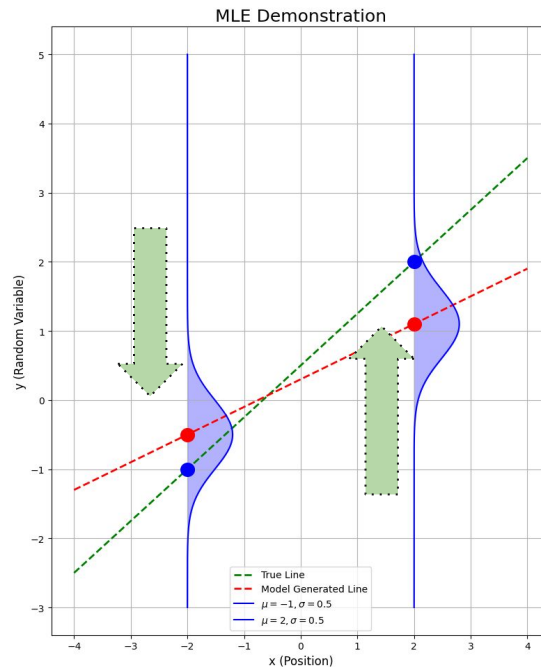
Probabilistic Twin

Probabilistic Formulation: Modeling Error Distribution

$$p(\mathbf{t}|\mathbf{x}, \mathbf{w}, \beta) = \prod_{n=1}^N \mathcal{N}(t_n | y(x_n, \mathbf{w}), \beta^{-1}).$$

Taking the log

$$\ln p(\mathbf{t}|\mathbf{x}, \mathbf{w}, \beta) = -\frac{\beta}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{N}{2} \ln \beta - \frac{N}{2} \ln(2\pi).$$



Probabilistic Twin

Probabilistic Formulation: Modeling Error Distribution

$$p(\mathbf{t}|\mathbf{x}, \mathbf{w}, \beta) = \prod_{n=1}^N \mathcal{N}(t_n | y(x_n, \mathbf{w}), \beta^{-1}).$$

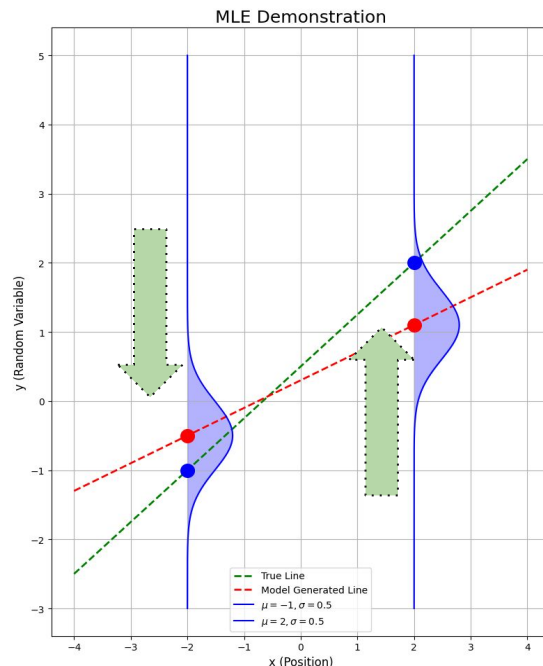
Taking the log

$$\ln p(\mathbf{t}|\mathbf{x}, \mathbf{w}, \beta) = -\frac{\beta}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{N}{2} \ln \beta - \frac{N}{2} \ln(2\pi).$$

Does it look familiar???

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2$$

Maximizing Log likelihood is equivalent to minimizing the quadratic loss/error in the context of LR!





In this task, we ask you to fit a **second-degree polynomial regression model** of the form

$$y = \beta_0 + \beta_1 x + \beta_2 x^2$$

data file:
https://github.com/mdkamrulhasan/data-public/blob/main/miscellaneous/second_degree_polynomial_regression_data.csv

—

- Use the **quadratic loss function** (i.e., squared error).
- Use the learning rate $L = 0.0001$.
- Initialize model parameters as $\Theta_0 = \{\beta_0 = 0.0, \beta_1 = 0.0, \beta_2 = 1.0\}$.

[illegible]





General ML models

Supervised

- kNN
- Linear Regression
- Decision Tree
- Meta learners
 - Random Forest Regressor
 - Boosting Regressor
- Support Vector Regressor (SVRs)

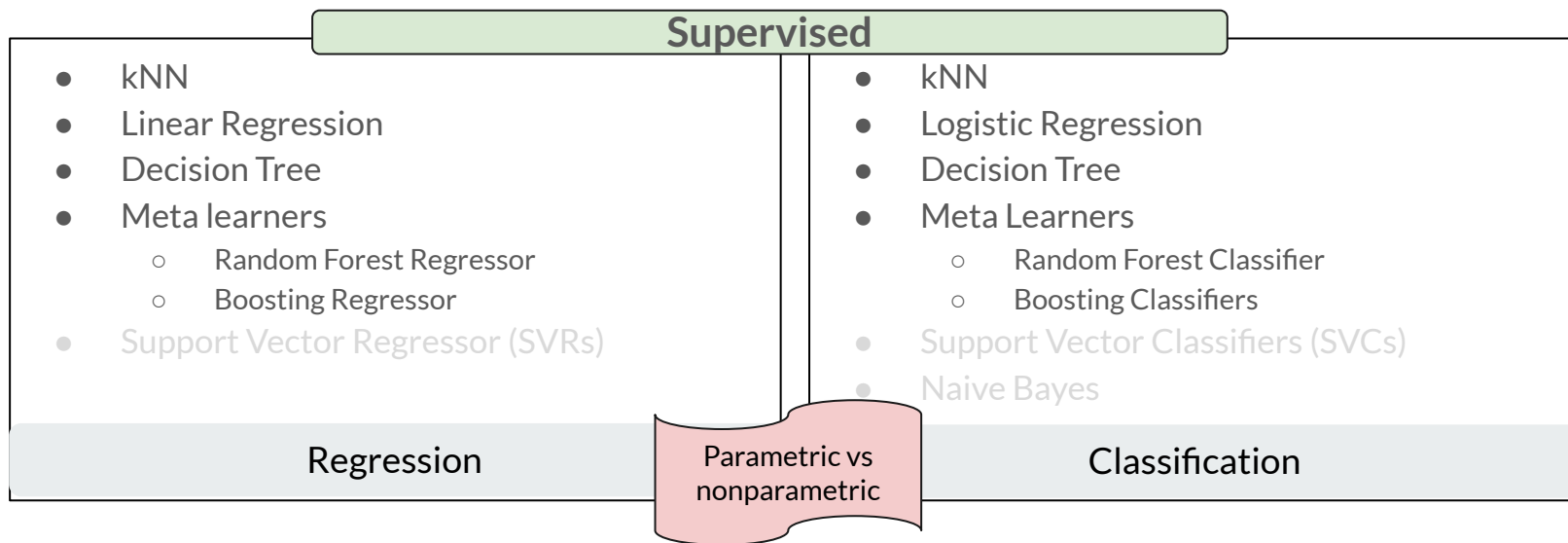
Regression

- kNN
- Logistic Regression
- Decision Tree
- Meta Learners
 - Random Forest Classifier
 - Boosting Classifiers
- Support Vector Classifiers (SVCs)
- Naive Bayes

Classification



General ML models





General ML models

Model generalization

- Universal concepts (applies to all models)
 - Cross validation
 - HP optimization

Universal concepts

- Overfitting
- Underfitting

Overfitting vs Under fitting



General ML models

Model generalization

- Training set, Validation set, Test set
- iid data

Training

Test /
Hold out set

Data splits

- Overfitting
- Underfitting

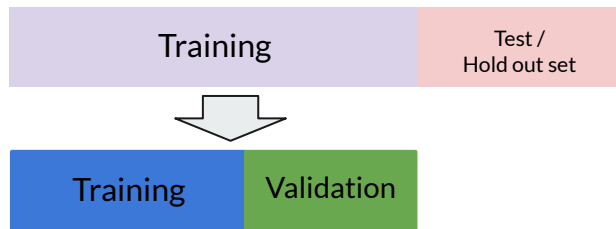
Overfitting vs Under fitting



General ML models

Model generalization

- Training set, Validation set, Test set
- iid data

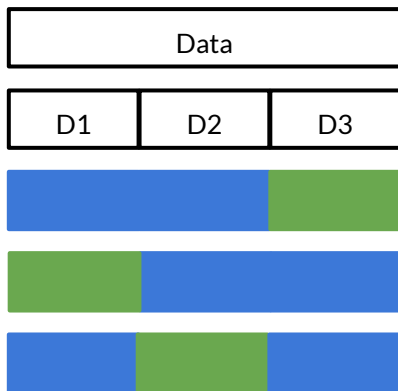


Data splits

- Overfitting
- Underfitting

Overfitting vs Under fitting

K-fold-cross validation



3-fold-cv

Train

validate

What HP gives the best validation score?

Task 1: Gradient Descent for Polynomial Regression

In this task, we ask you to fit a **second-degree polynomial regression model** of the form

$$y = \beta_0 + \beta_1 x + \beta_2 x^2,$$

where $\Theta = \{\beta_0, \beta_1, \beta_2\}$ denotes the model parameters including bias, β_0 . You should follow the instructions provided in the “Setup Details” section and apply the model to the dataset given below.

data file:

https://github.com/mdkamrulhasan/data-public/blob/main/miscellaneous/second_degree_polynomial_regression_data.csv

Setup Details:

- Use the **quadratic loss function** (i.e., squared error).
- Use the learning rate $L = 0.0001$.
- Initialize model parameters as $\Theta_0 = \{\beta_0 = 0.0, \beta_1 = 0.0, \beta_2 = 1.0\}$.

Task 2: Classification with Logistic Regression and Random Forest

In this task, you will build two classification models, **Logistic Regression** and **Random Forest**, and evaluate their performance using the metrics listed below.

Dataset:

https://github.com/mdkamrulhasan/data-public/blob/main/miscellaneous/adult_income_binary_processed.csv

[illegible]



Unsupervised Learning

Supervised Learning

- We learned about **Classification** and **Regression**
- These are examples of **supervised** learning
- In your data you have both **X(features)** and **y(Labels)**

$$f(y|X)$$

Label (y) is predefined

Classification

X	$y \in \{cat, dog, rabbit\}$
$x_1, x_2, \dots x_m$	cat
$x_1, x_2, \dots x_m$	rabbit
...	...
$x_1, x_2, \dots x_m$	dog

X	$y \in R$
$x_1, x_2, \dots x_n$	1.9
...	...
$x_1, x_2, \dots x_n$	2.5

Regression



Unsupervised Learning

- In contrast, in **Unsupervised learning**, we have to learn meaningful **representations/models** from **X(features)** only.
- **Clustering** is an example of unsupervised learning

X
$x_1, x_2, \dots x_o$
$\dots$
$x_1, x_2, \dots x_o$

No
concept
of data
label (y)



Unsupervised Learning

- In contrast, in **Unsupervised learning**, we have to learn meaningful **representations/models** from **X(features)** only.
- **Clustering** is an example of unsupervised learning

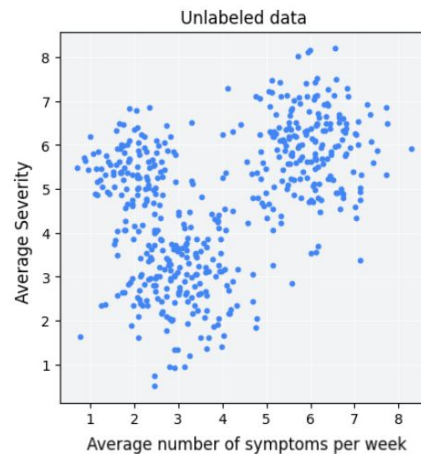
X
$x_1, x_2, \dots x_o$
$\dots$
$x_1, x_2, \dots x_o$

No
concept
of data
label (y)

$f(X)$

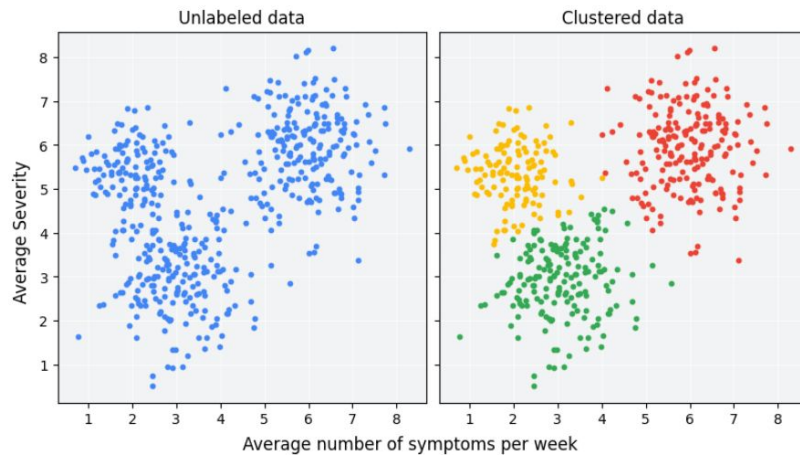
Clustering

- A medical study involving
 - Average number of symptoms per week
 - Average severity



Clustering

- A medical study involving
 - Average number of symptoms per week
 - Average severity



Usages of Clustering

Clustering has a myriad of uses in a variety of industries. Some common applications for clustering include the following:

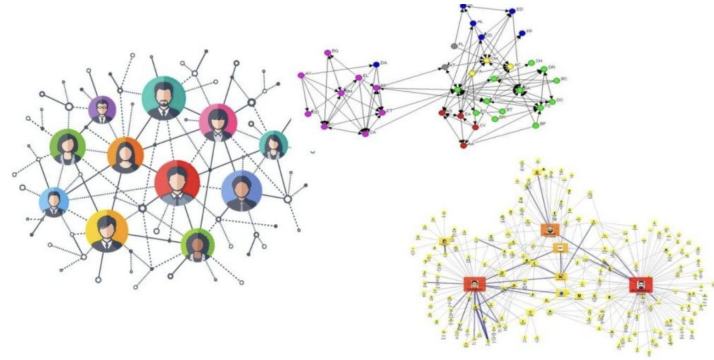
- **Market segmentation**
- Social network analysis
- Search result grouping
- Medical imaging
- Image segmentation
- Anomaly detection



Usages of Clustering

Clustering has a myriad of uses in a variety of industries. Some common applications for clustering include the following:

- Market segmentation
- **Social network analysis**
- Search result grouping
- Medical imaging
- Image segmentation
- Anomaly detection



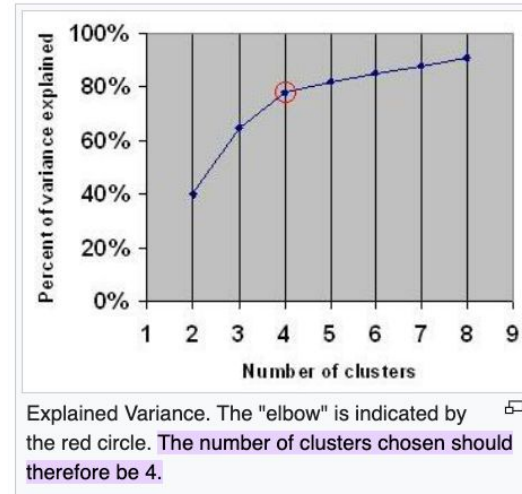


Clustering

- What is clustering?
- Clustering algorithms:
 - **K-Means:** Centroid Based
 - **Hierarchical clustering:** Distance connectivity based
 - **GMM:** Distribution based
 - **DBSCAN:** Density Based
- Identifying the number of clusters ?

Selecting the Number of clusters

- Elbow method
- Silhouette score based

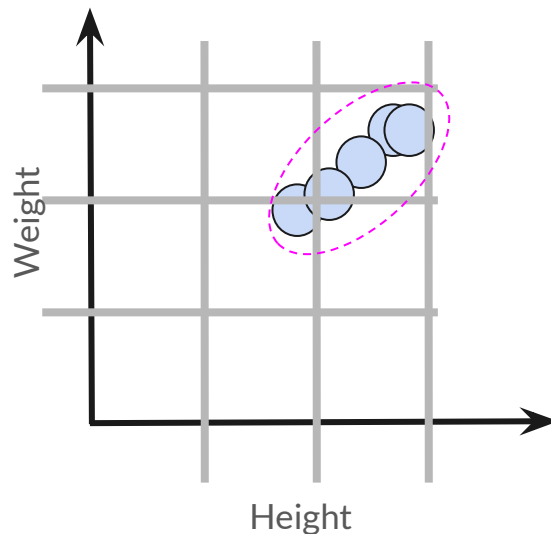




Curse of Dimensionality

Curse of Dimensionality

- We have added Weight as our second feature dimension
- Do you find the samples represent our class?
- What's the domain coverage now?
- Is ~10% a reasonable assumption?
- See how numbers dropped from ~40% to ~10%
- The other way, the empty space grew up from ~60% to ~90%
- **What's the number would look like if we add 2 more dimensions?**



- Empty space grows exponentially with the increase in adding new features.
- **Data distribution becomes sparse, and difficult to learn a good model.**



Dimensionality Reduction

General idea

- You have some data of feature dimension size, $|D|$
- You want to compress them to of size, $|d|$
- $|d| < |D|$



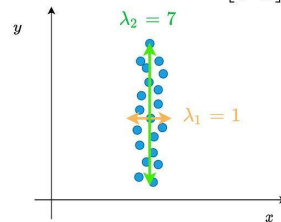
Dimensionality Reduction

- Linear
 - Principal Component Analysis (PCA)
 - SVD
- Neural Networks
 - Auto Encoders
 - RBMs

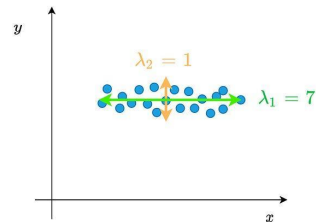
PCA

Notebook presentation

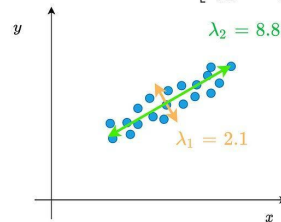
1 $C = \begin{bmatrix} 1 & 0 \\ 0 & 7 \end{bmatrix}$ $\lambda_{1,2} = [1 \ 7]$
 $V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$



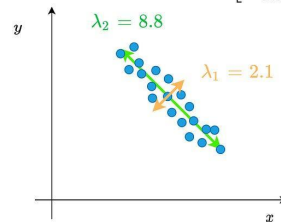
2 $C = \begin{bmatrix} 7 & 0 \\ 0 & 1 \end{bmatrix}$ $\lambda_{1,2} = [7 \ 1]$
 $V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$



3 $C = \begin{bmatrix} 4 & 3 \\ 3 & 7 \end{bmatrix}$ $\lambda_{1,2} = [2.1 \ 8.8]$
 $V = \begin{bmatrix} -0.8 & -0.5 \\ 0.5 & -0.8 \end{bmatrix}$



4 $C = \begin{bmatrix} 4 & -3 \\ -3 & 7 \end{bmatrix}$ $\lambda_{1,2} = [2.1 \ 8.8]$
 $V = \begin{bmatrix} -0.8 & 0.5 \\ -0.5 & -0.8 \end{bmatrix}$



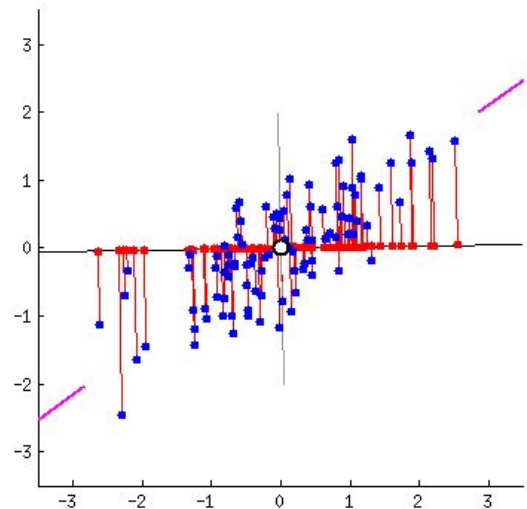
λ = eigenvalues

V = eigenvectors

What are we optimizing?

Two techniques (in general):

- Maximize variance
- Minimize errors (distance between each green-red paris)



ref



Task 1: Gradient Descent for Polynomial Regression

In this task, we ask you to fit a **second-degree polynomial regression model** of the form

$$y = \beta_0 + \beta_1 x + \beta_2 x^2,$$

where $\Theta = \{\beta_0, \beta_1, \beta_2\}$ denotes the model parameters including bias, β_0 . You should follow the instructions provided in the “Setup Details” section and apply the model to the dataset given below.

data file:
https://github.com/mdkamrulhasan/data-public/blob/main/miscellaneous/second_degree_polynomial_regression_data.csv

Setup Details:

- Use the **quadratic loss function** (i.e., squared error).
- Use the learning rate $L = 0.0001$.
- Initialize model parameters as $\Theta_0 = \{\beta_0 = 0.0, \beta_1 = 0.0, \beta_2 = 1.0\}$.

Task 2: Classification with Logistic Regression and Random Forest

In this task, you will build two classification models, **Logistic Regression** and **Random Forest**, and evaluate their performance using the metrics listed below.

Dataset:

https://github.com/mdkamrulhasan/data-public/blob/main/miscellaneous/adult_income_binary_processed.csv

[illegible]





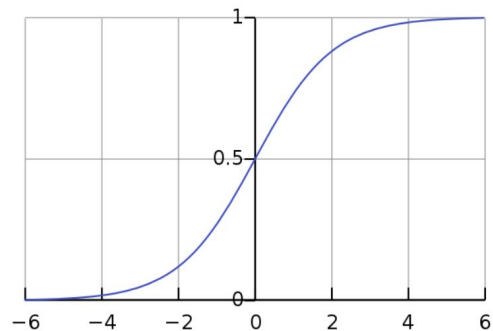
[illegible]

Logistic Regression

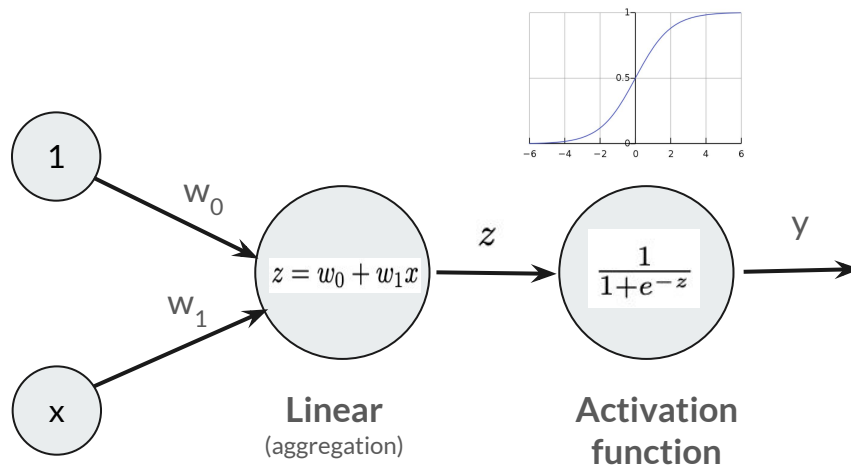
- Probabilistic classifier

$$p(x) = \frac{1}{1 + e^{-(w_0 + w_1 x)}}$$

- Sigmoid function

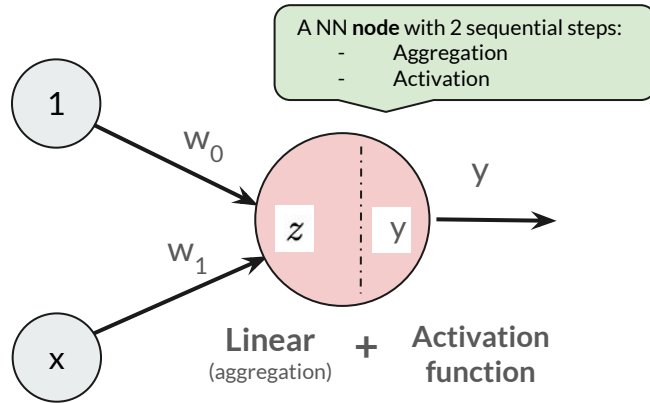


Neural Networks (Node)



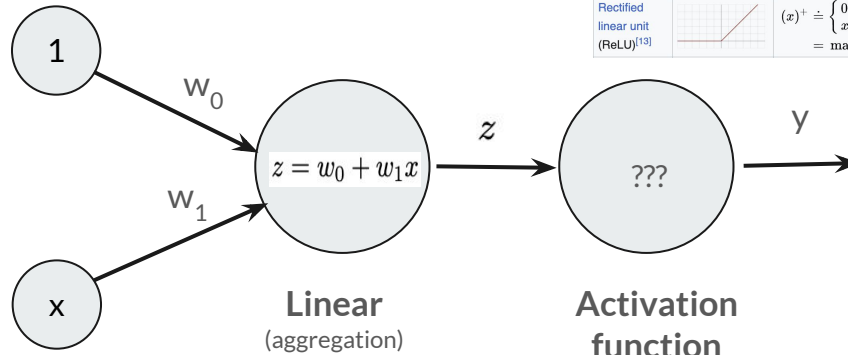
$y \in \{0, 1\}$
Logistic Regression

Neural Networks (Node)



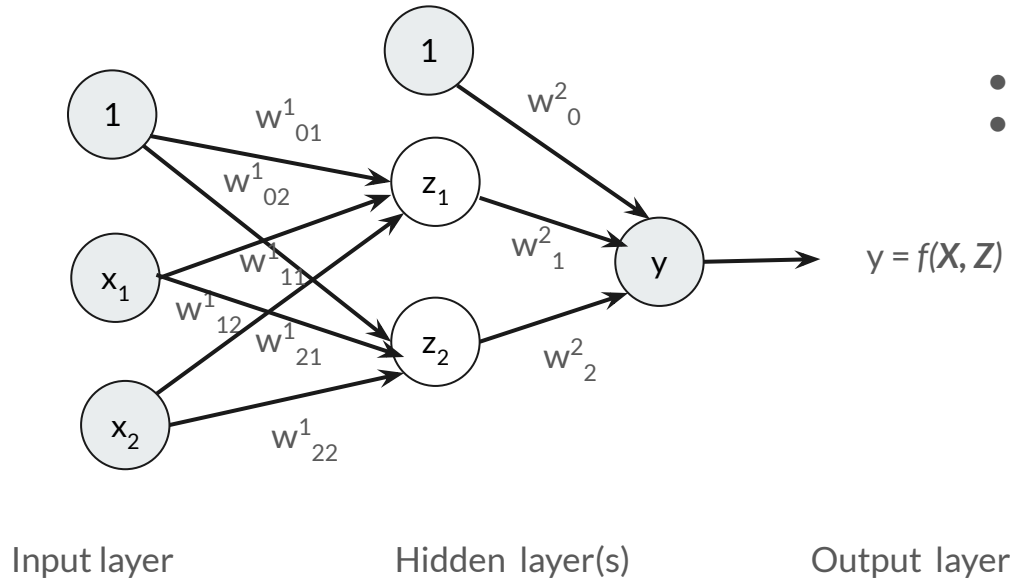
Neural Networks (Node)

Name	Plot	Function, $g(x)$
Identity		x
Binary step		$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$
Logistic, sigmoid, or soft step		$\sigma(x) \doteq \frac{1}{1 + e^{-x}}$
Hyperbolic tangent (tanh)		$\tanh(x) \doteq \frac{e^x - e^{-x}}{e^x + e^{-x}}$
Soboleva modified hyperbolic tangent (smht)		$\text{smht}(x) \doteq \frac{e^{ax} - e^{-bx}}{e^{cx} + e^{-dx}}$
Rectified linear unit (ReLU) ^[13]		$(x)^+ \doteq \begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$ $= \max(0, x) = x \mathbf{1}_{x>0}$



A NN with ???
Activation function

Feed-forward (FF) neural networks



- Feedforward NN
- Perceptron



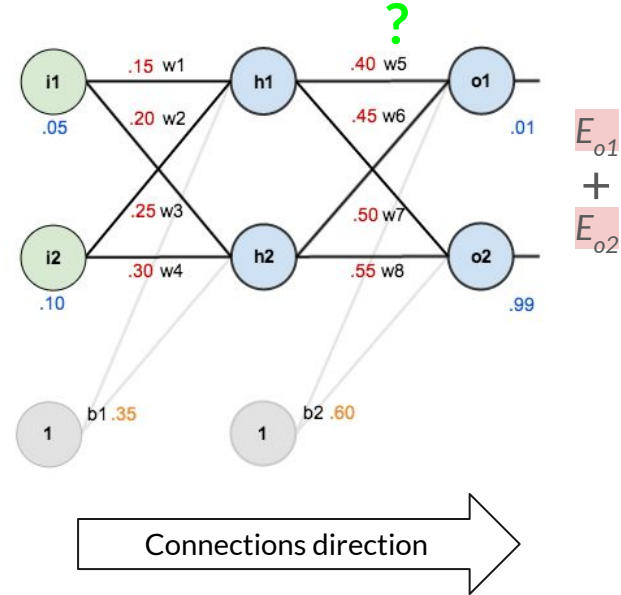
Error Back propagation

Gradient Descent (Error Back Propagation)

The Backwards Pass

Let's focus on $\frac{\partial E_{total}}{\partial w_5}$

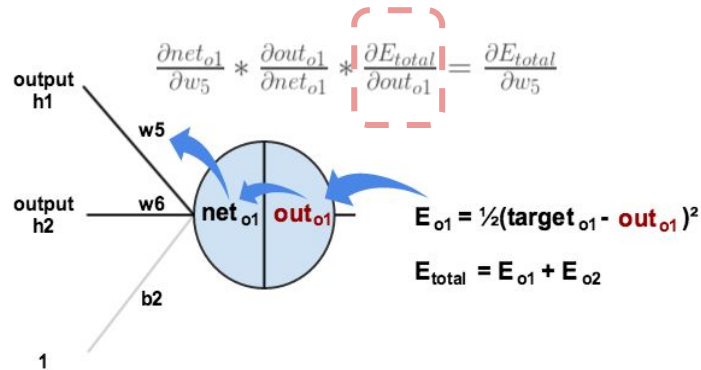
What would be the gradient update for w_5 ?



Adapted from

Gradient Descent (Error Back Propagation)

The Backwards Pass



$$E_{total} = \frac{1}{2}(\text{target}_{o1} - \text{out}_{o1})^2 + \frac{1}{2}(\text{target}_{o2} - \text{out}_{o2})^2$$

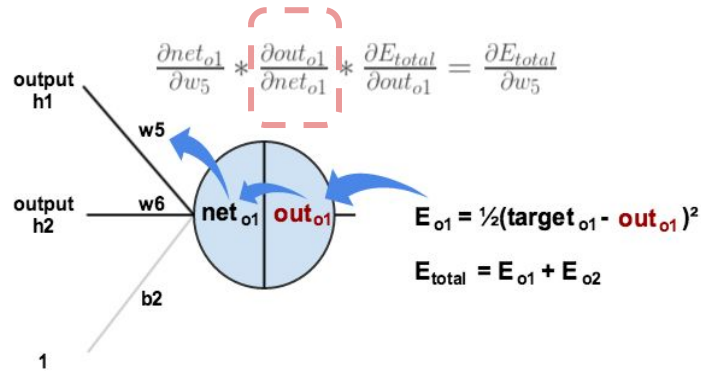
$$\frac{\partial E_{total}}{\partial out_{o1}} = 2 * \frac{1}{2}(\text{target}_{o1} - \text{out}_{o1})^{2-1} * -1 + 0$$

$$\frac{\partial E_{total}}{\partial out_{o1}} = -(\text{target}_{o1} - \text{out}_{o1}) = -(0.01 - 0.75136507) = 0.74136507$$

Adapted from

Gradient Descent (Error Back Propagation)

The Backwards Pass



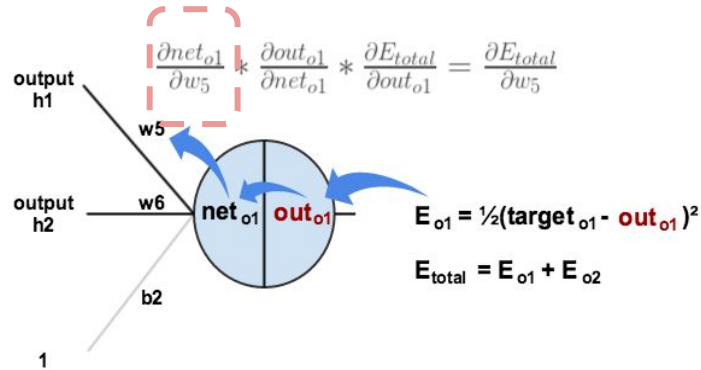
$$out_{o1} = \frac{1}{1 + e^{-net_{o1}}}$$

$$\begin{aligned}\frac{\partial out_{o1}}{\partial net_{o1}} &= out_{o1}(1 - out_{o1}) \\ &= 0.75136507(1 - 0.75136507) \\ &= 0.186815602\end{aligned}$$

Adapted from

Gradient Descent (Error Back Propagation)

The Backwards Pass



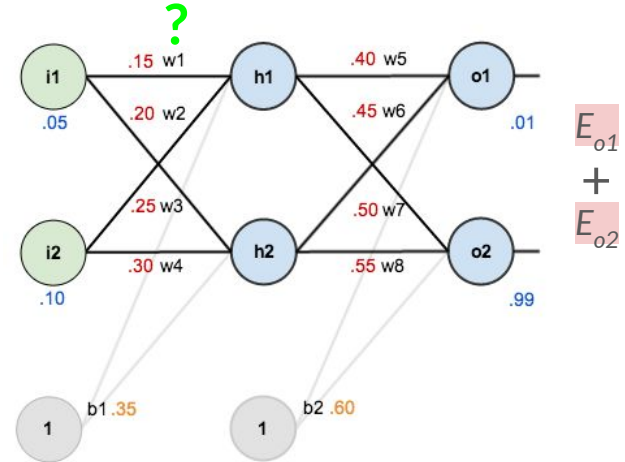
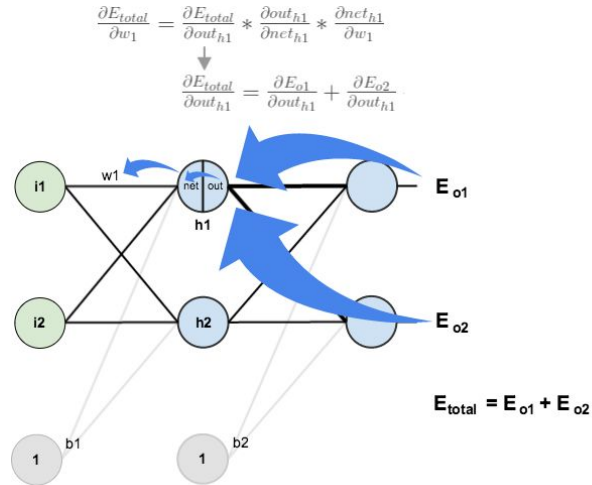
$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

$$\frac{\partial net_{o1}}{\partial w_5} = 1 * out_{h1} * w_5^{(1-1)} + 0 + 0 = out_{h1} = 0.593269992$$

Adapted from

Gradient Descent (Error Back Propagation)

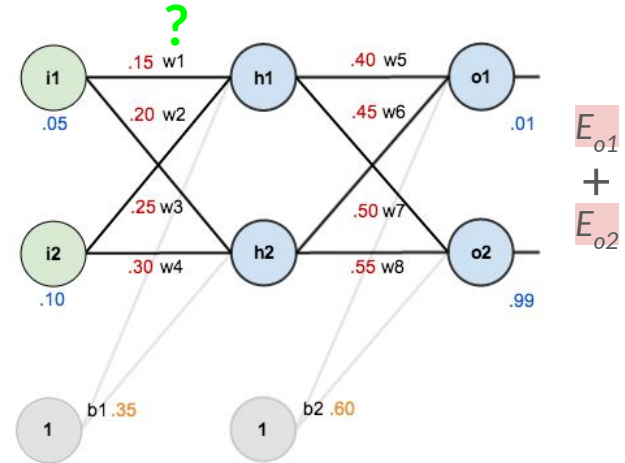
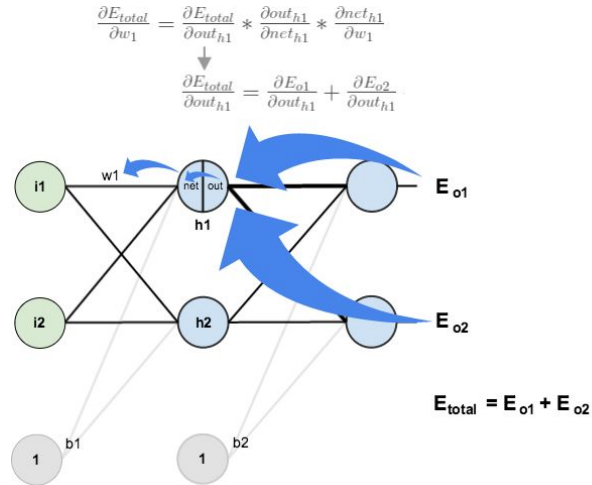
Hidden Layer



- While changing w_5 affects only O_1 , a change in w_1 will change both O_1 and O_2

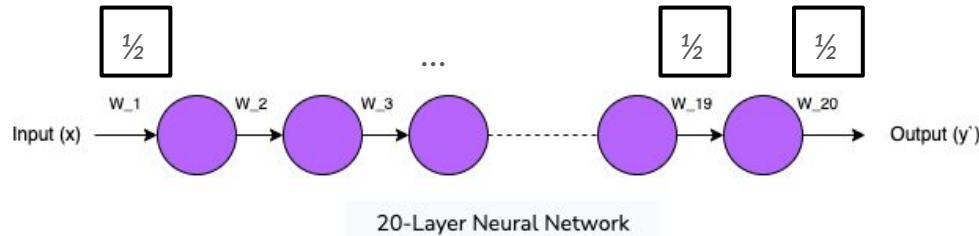
Gradient Descent (Error Back Propagation)

Hidden Layer



- Can you think of an arbitrary node in a giant and complex NN? What challenges we may encounter?

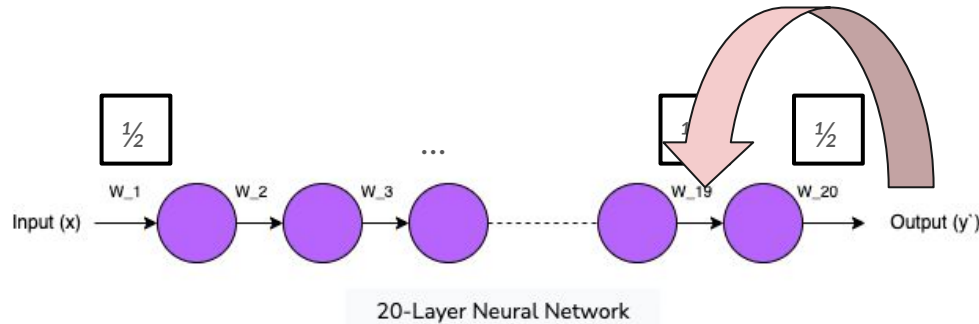
Vanishing and Exploding Gradients



Imagine if each step partial derivative is : $\frac{1}{2}$

$$\boxed{o_1 = a_1(w_1 \cdot x)} \quad \boxed{o_2 = a_2(w_2 \cdot a_1(w_1 \cdot x))} \quad \dots \quad (l=20)$$

Vanishing and Exploding Gradients



Imagine if each step partial derivative is : $\frac{1}{2}$

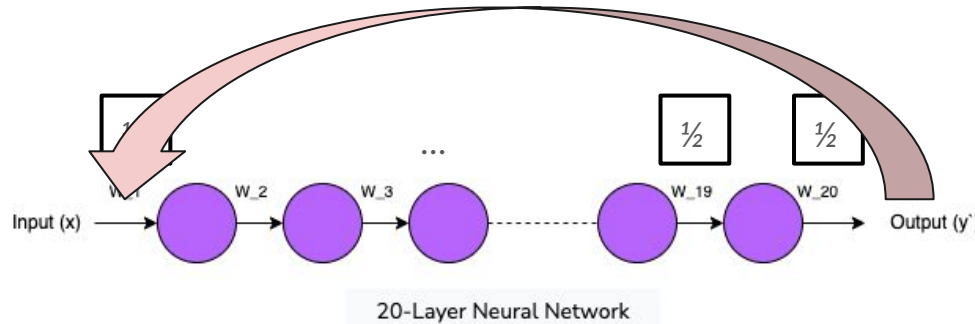
What would be the partial gradient at the first step (w_1) at the error back-propagation step:

$$o_1 = a_1(w_1 \cdot x)$$

$$o_2 = a_2(w_2 \cdot a_1(w_1 \cdot x))$$

..... ($l=20$)

Vanishing and Exploding Gradients



Imagine if each step partial derivative is : $\frac{1}{2}$

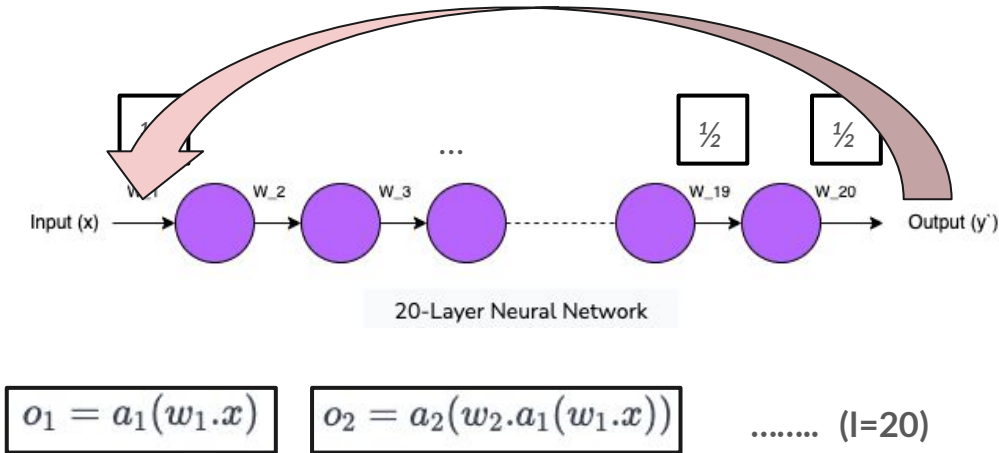
What would be the partial gradient at the first step (w_1) at the error back-propagation step:

$$o_1 = a_1(w_1 \cdot x)$$

$$o_2 = a_2(w_2 \cdot a_1(w_1 \cdot x))$$

..... ($l=20$)

Vanishing and Exploding Gradients



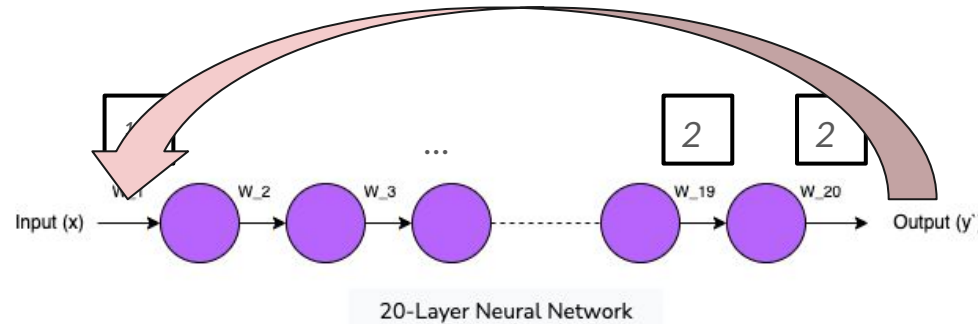
Imagine if each step partial derivative is : $\frac{1}{2}$

What would be the partial gradient at the first step (w_1) at the error back-propagation step:

$$\left(\frac{1}{2}\right)^{20}$$

Vanishing Gradient Problem

Vanishing and Exploding Gradients



$$o_1 = a_1(w_1 \cdot x)$$

$$o_2 = a_2(w_2 \cdot a_1(w_1 \cdot x))$$

..... (l=20)

Imagine if each step partial derivative is : 2

What would be the partial gradient at the first step (w_1) at the error back-propagation step:

$$(2)^{20}$$

Exploding Gradient Problem



QA